

3.2 动态规划算法设计

➤ 动态规划两种实现的比较

- **递归实现**：时间复杂性高，空间较小
- **迭代实现**：时间复杂性低，空间消耗多

➤ 原因：

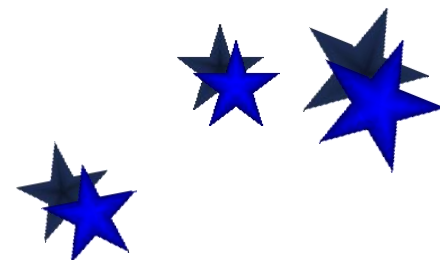
- 递归实现**子问题多次重复计算**，子问题计算次数呈指数增长。
 - 迭代实现**每个子问题只计算一次**。
- 动态规划时间复杂度：**备忘录各项计算量之和 + 追踪解工作量**。通常追踪工作量不超过备忘录计算的工作量，是问题规模的多项式函数。



3.2 动态规划算法设计

➤ 动态规划算法的要素：

- 划分子问题阶段，确定每个阶段中包含的状态结点。
- 确定子问题边界——最小规模问题，可直接求解。
- 列出各阶段间的递推方程和边界条件
- 自底向上计算每个阶段各结点状态的最优值。
- 设计备忘录 (表格)记录各状态结点的最优值， 考虑是否需要设立标记函数（用于追踪解的数据结构）。
- 用递推方程或备忘录估计时间复杂度



3.3 动态规划算法的典型应用

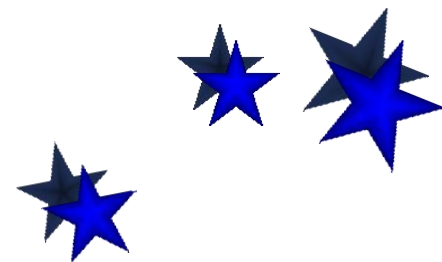
■ **投资问题**——资源分配问题：是将数量一定的一种或若干种资源（原材料、资金、设备或劳动力等），合理地分配给若干使用者，使总收益最大。

◆ 投资问题的建模

问题： m 元钱， n 项投资， $f_i(x)$ ：将 x 元投入第 i 个项目的效益。求使得总效益最大的投资方案。

建模：

问题的解是向量 $\langle x_1, x_2, \dots, x_n \rangle$ ，
 x_i 是投给项目 i 的钱数， $i=1, 2, \dots, n$.



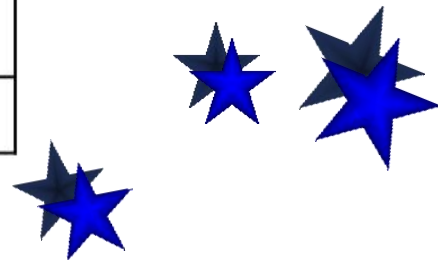
3.3 动态规划算法的典型应用

目标函数 $\max\{f_1(x_1)+f_2(x_2)+\dots+f_n(x_n)\}$

约束条件 $x_1+x_2+\dots+x_n=m, x_i \in \mathbb{N}$

◆ 实例： 5万元钱， 4个项目， 效益函数如下表所示：

x	$f_1(x)$	$f_2(x)$	$f_3(x)$	$f_4(x)$
0	0	0	0	0
1	11	0	2	20
2	12	5	10	21
3	13	10	30	22
4	14	15	32	23
5	15	20	40	24



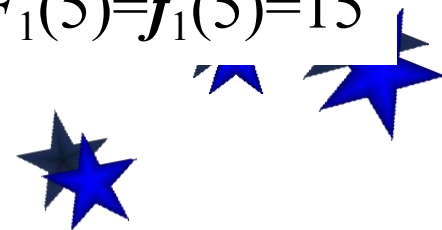
3.3 动态规划算法的典型应用

➤ 实例分析——分阶段决策

(1) 第1阶段：分别计算将 $x(0 \leq x \leq 5)$ 万元分配给前1个项目的最大收益

- 分配0万元给前1个项目的最大收益为0，即 $F_1(0)=f_1(0)=0$
- 分配1万元给前1个项目的最大收益为11，即 $F_1(1)=f_1(1)=11$
- 分配2万元给前1个项目的最大收益为12，即 $F_1(2)=f_1(2)=12$
- 分配3万元给前1个项目的最大收益为13，即 $F_1(3)=f_1(3)=13$
- 分配4万元给前1个项目的最大收益为14，即 $F_1(4)=f_1(4)=14$
- 分配5万元给前1个项目的最大收益为15，即 $F_1(5)=f_1(5)=15$

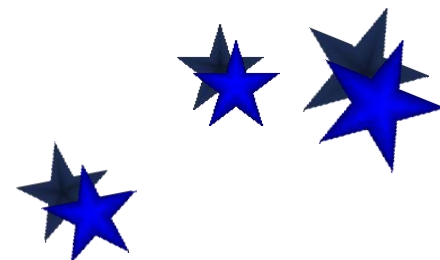
$$F_1=\{0,11,12,13,14,15\}$$



3.3 动态规划算法的典型应用

(2) 第2阶段：分别计算将 $x(0 \leq x \leq 5)$ 万元分配给前2个项目的最大收益

- 分配0万元给前2个项目的分配方案为：2项目0万元，前1个项目0万元，则最大收益 $F_2(0) = \max\{f_2(0), F_1(0)\} = \{0, 0\} = 0$
- 分配1万元给前2个项目的分配方案为：
 - ✓ 2项目0万元，前1个项目1万元，收益为： $f_2(0) + F_1(1) = 11$
 - ✓ 2项目1万元，前1个项目0万元，收益为： $f_2(1) + F_1(0) = 0$
 - ✓ 则最大收益 $F_2(1) = \max\{11, 0\} = 11$

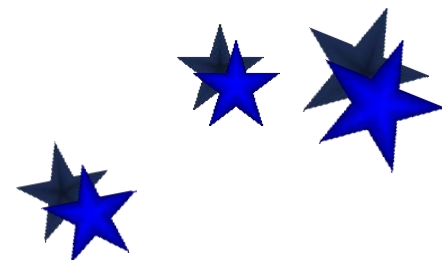


3.3 动态规划算法的典型应用

- 分配2万元给前2个项目的分配方案为：
 - ✓ 2项目0万元，前1个项目2万元，收益为： $f_2(0)+F_1(2)=12$
 - ✓ 2项目1万元，前1个项目1万元，收益为： $f_2(1)+F_1(1)=11$
 - ✓ 2项目2万元，前1个项目0万元，收益为： $f_2(2)+F_1(0)=2$
 - ✓ 则最大收益 $F_2(2)=\max\{12,11,2\}=12$
- 分配3万元给前2个项目的分配方案为：
 - ✓ 2项目0万元，前1个项目3万元，收益为： $f_2(0)+F_1(3)=13$
 - ✓ 2项目1万元，前1个项目2万元，收益为： $f_2(1)+F_1(2)=12$
 - ✓ 2项目2万元，前1个项目1万元，收益为： $f_2(2)+F_1(1)=16$
 - ✓ 2项目3万元，前1个项目0万元，收益为： $f_2(3)+F_1(0)=10$
 - ✓ 则最大收益 $F_2(3)=\max\{13,12,16,10\}=16$

3.3 动态规划算法的典型应用

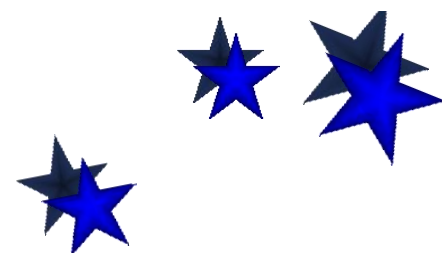
- 分配4万元给前2个项目的分配方案为：
 - ✓ 2项目0万元，前1个项目4万元，收益为： $f_2(0)+F_1(4)=14$
 - ✓ 2项目1万元，前1个项目3万元，收益为： $f_2(1)+F_1(3)=13$
 - ✓ 2项目2万元，前1个项目2万元，收益为： $f_2(2)+F_1(2)=17$
 - ✓ 2项目3万元，前1个项目1万元，收益为： $f_2(3)+F_1(1)=21$
 - ✓ 2项目4万元，前1个项目0万元，收益为： $f_2(4)+F_1(0)=15$
 - ✓ 则最大收益 $F_2(4)=\max\{14,13,17,21,15\}=21$



3.3 动态规划算法的典型应用

- 分配5万元给前2个项目的分配方案为：
 - ✓ 2项目0万元，前1个项目5万元，收益为： $f_2(0)+F_1(5)=15$
 - ✓ 2项目1万元，前1个项目4万元，收益为： $f_2(1)+F_1(4)=14$
 - ✓ 2项目2万元，前1个项目3万元，收益为： $f_2(2)+F_1(3)=18$
 - ✓ 2项目3万元，前1个项目2万元，收益为： $f_2(3)+F_1(2)=22$
 - ✓ 2项目4万元，前1个项目1万元，收益为： $f_2(4)+F_1(1)=26$
 - ✓ 2项目5万元，前1个项目0万元，收益为： $f_2(5)+F_1(0)=20$
 - ✓ 则最大收益 $F_2(4)=\max\{15,14,18,22,26,20\}=26$

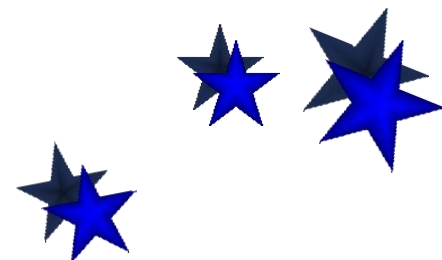
$$F_2=\{0,11,12,16,21,26\}$$



3.3 动态规划算法的典型应用

(3) 第3阶段：分别计算将 $x(0 \leq x \leq 5)$ 万元分配给前3个项目的最大收益

- 分配0万元给前3个项目的分配方案为：3项目0万元，前2个项目0万元，则最大收益 $F_3(0) = \max\{f_3(0), F_2(0)\} = \{0, 0\} = 0$
- 分配1万元给前3个项目的分配方案为：
 - ✓ 3项目0万元，前2个项目1万元，收益为： $f_3(0) + F_2(1) = 11$
 - ✓ 3项目1万元，前2个项目0万元，收益为： $f_3(1) + F_2(0) = 2$
 - ✓ 则最大收益 $F_3(1) = \max\{11, 2\} = 11$

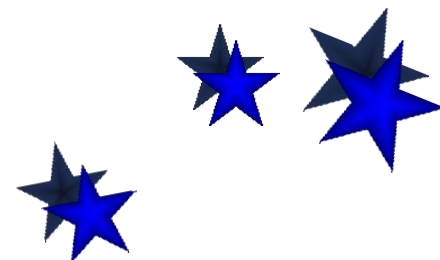


3.3 动态规划算法的典型应用

- 分配2万元给前3个项目的分配方案为：
 - ✓ 3项目0万元，前2个项目2万元，收益为： $f_3(0)+F_2(2)=12$
 - ✓ 3项目1万元，前2个项目1万元，收益为： $f_3(1)+F_2(1)=13$
 - ✓ 3项目2万元，前2个项目0万元，收益为： $f_3(2)+F_2(0)=10$
 - ✓ 则最大收益 $F_3(2)=\max\{12,13,10\}=13$
- 分配3万元给前3个项目的分配方案为：
 - ✓ 3项目0万元，前2个项目3万元，收益为： $f_3(0)+F_2(3)=16$
 - ✓ 3项目1万元，前2个项目2万元，收益为： $f_3(1)+F_2(2)=14$
 - ✓ 3项目2万元，前2个项目1万元，收益为： $f_3(2)+F_2(1)=21$
 - ✓ 3项目3万元，前2个项目0万元，收益为： $f_3(3)+F_2(0)=30$
 - ✓ 则最大收益 $F_3(3)=\max\{16,14,21,30\}=30$

3.3 动态规划算法的典型应用

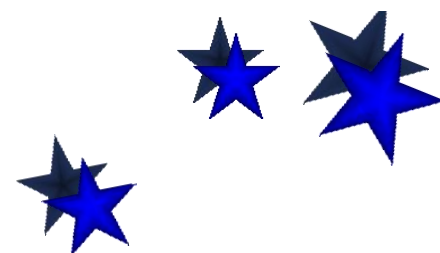
- 分配4万元给前3个项目的分配方案为：
 - ✓ 3项目0万元，前2个项目4万元，收益为： $f_3(0)+F_2(4)=21$
 - ✓ 3项目1万元，前2个项目3万元，收益为： $f_3(1)+F_2(3)=18$
 - ✓ 3项目2万元，前2个项目2万元，收益为： $f_3(2)+F_2(2)=22$
 - ✓ 3项目3万元，前2个项目1万元，收益为： $f_3(3)+F_2(1)=41$
 - ✓ 3项目4万元，前2个项目0万元，收益为： $f_3(4)+F_2(0)=32$
 - ✓ 则最大收益 $F_3(4)=\max\{21,18,22,41,32\}=41$



3.3 动态规划算法的典型应用

- 分配5万元给前3个项目的分配方案为：
 - ✓ 3项目0万元，前2个项目5万元，收益为： $f_3(0)+F_2(5)=26$
 - ✓ 3项目1万元，前2个项目4万元，收益为： $f_3(1)+F_2(4)=23$
 - ✓ 3项目2万元，前2个项目3万元，收益为： $f_3(2)+F_2(3)=26$
 - ✓ 3项目3万元，前2个项目2万元，收益为： $f_3(3)+F_2(2)=42$
 - ✓ 3项目4万元，前2个项目1万元，收益为： $f_3(4)+F_2(1)=43$
 - ✓ 3项目5万元，前2个项目0万元，收益为： $f_3(5)+F_2(0)=40$
 - ✓ 则最大收益 $F_3(5)=\max\{26,23,26,42,43,40\}=43$

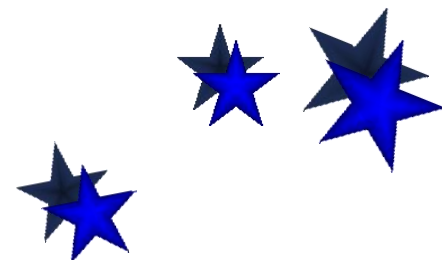
$$F_3=\{0,11,13,30,41,43\}$$



3.3 动态规划算法的典型应用

(4) 第4阶段：分别计算将 $x(0 \leq x \leq 5)$ 万元分配给前4个项目的最大收益

- 分配0万元给前4个项目的分配方案为：4项目0万元，前3个项目0万元，则最大收益 $F_4(0) = \max\{f_4(0), F_3(0)\} = \{0, 0\} = 0$
- 分配1万元给前4个项目的分配方案为：
 - ✓ 4项目0万元，前3个项目1万元，收益为： $f_4(0) + F_3(1) = 11$
 - ✓ 4项目1万元，前3个项目0万元，收益为： $f_4(1) + F_3(0) = 20$
 - ✓ 则最大收益 $F_4(1) = \max\{11, 20\} = 20$

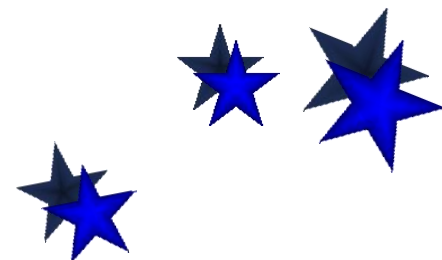


3.3 动态规划算法的典型应用

- 分配2万元给前4个项目的分配方案为：
 - ✓ 4项目0万元，前3个项目2万元，收益为： $f_4(0)+F_3(2)=13$
 - ✓ 4项目1万元，前3个项目1万元，收益为： $f_4(1)+F_3(1)=31$
 - ✓ 4项目2万元，前3个项目0万元，收益为： $f_4(2)+F_3(0)=21$
 - ✓ 则最大收益 $F_4(2)=\max\{13,31,21\}=31$
- 分配3万元给前4个项目的分配方案为：
 - ✓ 4项目0万元，前3个项目3万元，收益为： $f_4(0)+F_3(3)=30$
 - ✓ 4项目1万元，前3个项目2万元，收益为： $f_4(1)+F_3(2)=33$
 - ✓ 4项目2万元，前3个项目1万元，收益为： $f_4(2)+F_3(1)=32$
 - ✓ 4项目3万元，前3个项目0万元，收益为： $f_4(3)+F_3(0)=22$
 - ✓ 则最大收益 $F_4(3)=\max\{30,33,32,22\}=33$

3.3 动态规划算法的典型应用

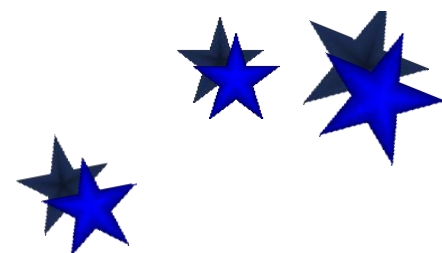
- 分配4万元给前4个项目的分配方案为：
 - ✓ 4项目0万元，前3个项目4万元，收益为： $f_4(0)+F_3(4)=41$
 - ✓ 4项目1万元，前3个项目3万元，收益为： $f_4(1)+F_3(3)=50$
 - ✓ 4项目2万元，前3个项目2万元，收益为： $f_4(2)+F_3(2)=34$
 - ✓ 4项目3万元，前3个项目1万元，收益为： $f_4(3)+F_3(1)=33$
 - ✓ 4项目4万元，前3个项目0万元，收益为： $f_4(4)+F_3(0)=23$
 - ✓ 则最大收益 $F_4(4)=\max\{41,50,34,33,23\}=50$



3.3 动态规划算法的典型应用

- 分配5万元给前4个项目的分配方案为：
 - ✓ 4项目0万元，前3个项目5万元，收益为： $f_4(0)+F_3(5)=43$
 - ✓ 4项目1万元，前3个项目4万元，收益为： $f_4(1)+F_3(4)=61$
 - ✓ 4项目2万元，前3个项目3万元，收益为： $f_4(2)+F_3(3)=51$
 - ✓ 4项目3万元，前3个项目2万元，收益为： $f_4(3)+F_3(2)=35$
 - ✓ 4项目4万元，前3个项目1万元，收益为： $f_4(4)+F_3(1)=34$
 - ✓ 4项目5万元，前3个项目0万元，收益为： $f_4(5)+F_3(0)=24$
 - ✓ 则最大收益 $F_4(5)=\max\{43,61,51,35,34,24\}=61$

$$F_4=\{0,20,31,33,50,61\}$$



3.3 动态规划算法的典型应用

➤ 解的追踪

(1) $F_4(5)$ 即为：前4个项目投资5万元的最大收益为61，即整个问题的解。

(2) 该解是给项目4投资了1万元，则前3个项目的投资为4万元。

(3) 在 $F_3(4)$ 的最大收益为41，该解是给项目3投资了3万元，则前2个项目的投资为1万元。

(4) 在 $F_2(1)$ 的最大收益为11，该解是给项目2投资了0万元，则前1个项目的投资为1万元。

(5) 在 $F_1(1)$ 的最大收益为11，该解是给项目1投资了1万元。



3.3 动态规划算法的典型应用

➤ 数据结构的设计

(1) 效益数组 f ，该数组的内容是逐行使用，在计算前 k 个项目时，只用 f_k 。因此可以使用一维数组逐行输入。

(2) 最大收益数组 F ，在计算 F_k 时，仅反复使用 F_{k-1} ，因此可以使用一维数组 dp 存储 F_{k-1} ，计算 F_k 过程中使用一维数组 $temp$ 暂存， k 阶段决策结束后，将 $temp$ 数组的内容送入 dp 。最后一阶段结束后， $dp[5]$ 中的值就是最优解。

(3) 标记函数——追踪解：资源分配数组 $bonus$ ， $bonus[i,j]$ 记录 i 阶段投资 j 万元获得最大收益时，分给项目 i 的投资金额。



3.3 动态规划算法的典型应用

➤ 根据数组内容给出决策结果

<i>dp</i>	0	1	2	3	4	5
	0	20	31	33	50	61

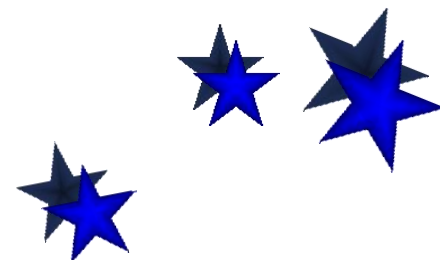
<i>bonus</i>		0	1	2	3	4	5
	1	0	1	2	3	4	5
	2	0	0	0	2	3	4
	3	0	0	1	3	3	4
	4	0	1	1	1	1	1



3.3 动态规划算法的典型应用

➤ 子问题界定和计算顺序

- 子问题界定：由参数 k 和 x 界定
 - (1) k : 考虑对前 $1, 2, \dots, k$ 个项目的投资——阶段
 - (2) x : 投资总钱数为 $x(0 \leq x \leq m)$
- 原始输入： $k = n, x = m$
 - (1) 子问题计算顺序： $k = 1, 2, \dots, n$
 - (2) 对于给定的 $k, x = 1, 2, \dots, m$



3.3 动态规划算法的典型应用

➤ 优化函数的递推方程

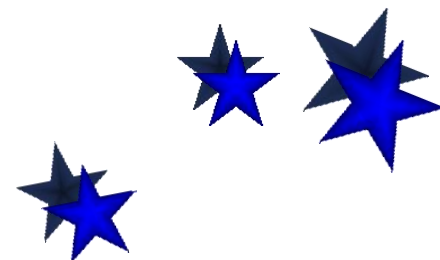
$F_k(x)$: x 元钱投给前 k 个项目最大效益

多步判断: 若知道 p 元钱 ($p \leq x$) 投给前 $k-1$ 个项目的最大效益 $F_{k-1}(p)$, 确定 x 元钱投给前 k 个项目的方案

递推方程和边界条件

$$F_k(x) = \max_{0 \leq x_k \leq x} \{f_k(x_k) + F_{k-1}(x - x_k)\} \quad k > 1$$

$$F_1(x) = f_1(x)$$



3.3 动态规划算法的典型应用

➤ 时间复杂度分析

备忘录表中有 m 行 n 列, 共计 mn 项

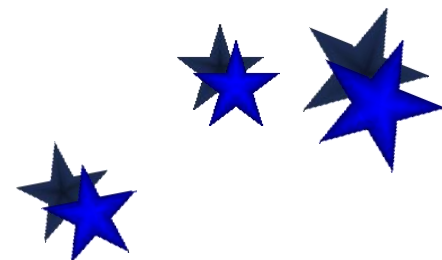
$$F_k(x) = \max_{0 \leq x_k \leq x} \{f_k(x_k) + F_{k-1}(x - x_k)\} \quad k > 1$$

$$F_1(x) = f_1(x)$$

x_k 有 $x+1$ 种可能的取值, 计算 $F_k(x)$ 项 ($2 \leq k \leq n, 1 \leq x \leq m$) 需要:

$x+1$ 次加法

x 次比较



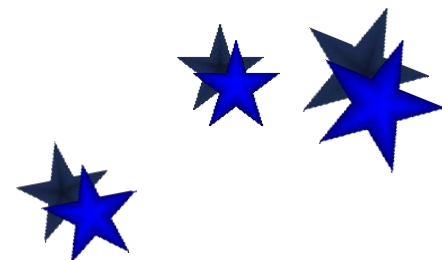
3.3 动态规划算法的典型应用

对备忘录中所有的项求和：

$$\text{加法次数} \sum_{k=2}^n \sum_{x=1}^m (x+1) = \frac{1}{2}(n-1)m(m+3)$$

$$\text{比较次数} \sum_{k=2}^n \sum_{x=1}^m x = \frac{1}{2}(n-1)m(m+1)$$

$$W(n) = O(nm^2)$$

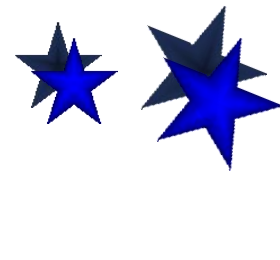


3.3 动态规划算法的典型应用

■ 背包问题

◆ 背包问题的分类

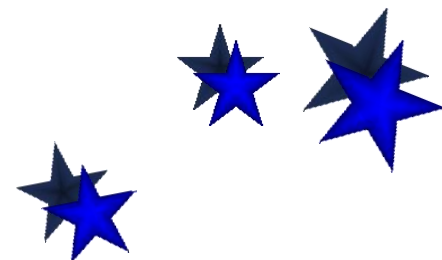
- (1) **0-1背包问题**: 物品 i 要么不装入背包(0), 要么就装1个(1)。
- (2) **完全背包问题**: 背包中物品 i 可以装入 x 个, x 为 ≥ 0 的整数。
- (3) 可拆包的背包问题: 按一定顺序装入背包的物品, 其中最后一个物品可按比例装入背包——贪心策略
- (4) 物品数受限背包: 第 i 种物品最多用 n_i 个。



3.3 动态规划算法的典型应用

(5) 多背包问题: m 个背包, 背包 j 装入最大重量 B_j , $j=1, 2, \dots, m$. 在满足所有背包重量约束条件下使装入物品价值最大。

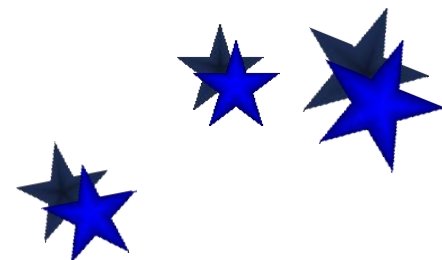
(6) 二维背包问题: 每件物品有重量 w_i 和体积 t_i , $i=1, 2, \dots, n$, 背包总重不超过 b , 体积不超过 V , 如何选择物品以得到最大价值。



3.3 动态规划算法的典型应用

◆ 0-1背包问题

- 问题描述：有 n 个重量分别为 $\{w_1, w_2, \dots, w_n\}$ 的物品，它们的价值分别为 $\{v_1, v_2, \dots, v_n\}$ ，给定一个容量为 W 的背包，设计从这些物品中选取一部分物品放入该背包，使重量和不超过 W 具有最大的价值。
- 解向量： $\langle x_1, x_2, \dots, x_n \rangle$ ，其中 $x_i=0$ 或 1 ，表示物品 i 是否装入背包。
- 目标函数： $\max \{x_1 v_1 + x_2 v_2 + x_3 v_3 + \dots + x_n v_n\}$
- 约束条件： $x_1 w_1 + x_2 w_2 + x_3 w_3 + \dots + x_n w_n \leq W$



3.3 动态规划算法的典型应用

- 阶段划分

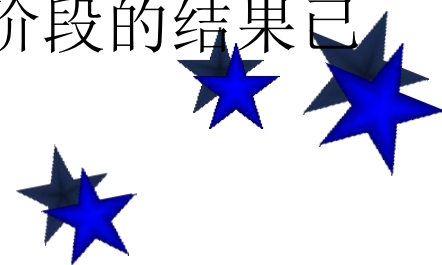
- (1) 根据物品的个数，将问题划分为 n 个阶段。
- (2) 第 i 阶段：背包容量在不超过 y ($0 \leq y \leq W$) 情况下，装入前 i ($0 \leq i \leq n$) 个物品的最大价值，表示为 $F_i(y)$ 。

- 边界条件

- (1) $F_i(0)=0$ 背包容量为0时，背包的总价值为0。
- (2) $F_0(y)=0$ 背包中没有装入物品时的总价值为0。

- 状态转移方程

- (1) 在 i 阶段：决策物品 i 是否装入时，前 $i-1$ 阶段的结果已经生成。



3.3 动态规划算法的典型应用

$$(2) F_i(y) = \max \{ F_{i-1}(y), F_{i-1}(y - w_i) + v_i \}$$

不装入
物品*i*

装入物品*i*时，前*i-1*个物
品的剩余空间。

- 解的追踪

(1) 使用一维数组 dp 存储第*i*阶段的决策结果。在第*i*阶段可先将决策结果暂存于 $temp$ 数组，该阶段结束后再更新 dp 数组中的内容。

(2) 使用二维数组 sel 存储每个阶段对物品*i*是否装入的决策。 $sel[i, j]$ 就是在背包容量不超过*j*时对物品*i*的选择：0或1。



3.3 动态规划算法的典型应用

- 实例分析：已知5个物品的重量 $w=\{2, 2, 6, 5, 4\}$ ；价值 $v=\{6, 3, 5, 4, 6\}$ 。背包容量 $W=10$ 。请使用动态规划策略求解该0-1背包问题。

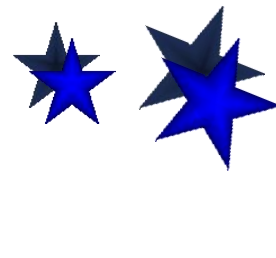
* 设计数据结构

(1) ***int dp[11];***//存储每个阶段的最优决策结果

(2) ***int sel[5][11];***//对物品 i 在容量不超过 j 的情况下决策结果（选1/不选0）。

* 初始化——背包没有装入物品时的价值

dp	0	1	2	3	4	5	6	7	8	9	10
	0	0	0	0	0	0	0	0	0	0	0



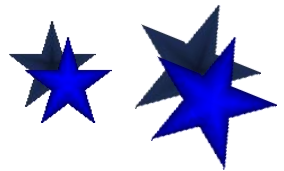
- * 第1阶段决策：前1个物品在背包容量不超过 j ($0 \leq j \leq 10$) 情况下的决策结果。 $w_1=2, v_1=6$

dp	0	1	2	3	4	5	6	7	8	9	10
	0	0	6	6	6	6	6	6	6	6	6

$$temp[j] = \max\{dp[j], dp[j-w_1] + v_1\} \quad 0 \leq j \leq 10$$

temp	0	1	2	3	4	5	6	7	8	9	10
	0	0	6	6	6	6	6	6	6	6	6

	0	1	2	3	4	5	6	7	8	9	10
1	0	0	1	1	1	1	1	1	1	1	1
2											
3											
4											
5											



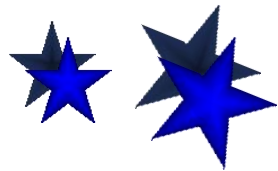
- * 第2阶段决策：前2个物品在背包容量不超过 j ($0 \leq j \leq 10$) 情况下的决策结果。 $w_2=2, v_2=3$

dp	0	1	2	3	4	5	6	7	8	9	10
	0	0	6	6	9	9	9	9	9	9	9

$$temp[j] = \max\{dp[j], dp[j-w_2] + v_2\} \quad 0 \leq j \leq 10$$

temp	0	1	2	3	4	5	6	7	8	9	10
	0	0	6	6	9	9	9	9	9	9	9

	0	1	2	3	4	5	6	7	8	9	10
1	0	0	1	1	1	1	1	1	1	1	1
2	0	0	0	0	1	1	1	1	1	1	1
3											
4											
5											



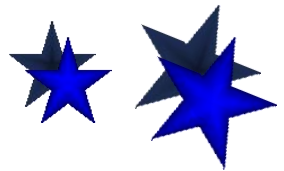
- * 第3阶段决策：前3个物品在背包容量不超过 j ($0 \leq j \leq 10$) 情况下的决策结果。 $w_3=6, v_3=5$

dp	0	1	2	3	4	5	6	7	8	9	10
	0	0	6	6	9	9	9	9	11	11	14

$$temp[j] = \max\{dp[j], dp[j-w_3] + v_3\} \quad 0 \leq j \leq 10$$

temp	0	1	2	3	4	5	6	7	8	9	10
	0	0	6	6	9	9	9	9	11	11	14

sel		0	1	2	3	4	5	6	7	8	9	10
	1	0	0	1	1	1	1	1	1	1	1	1
	2	0	0	0	0	1	1	1	1	1	1	1
	3	0	0	0	0	0	0	0	0	1	1	1
	4											
	5											



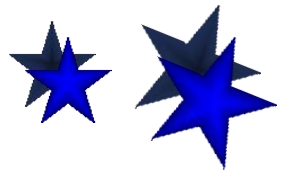
- * 第4阶段决策：前4个物品在背包容量不超过 j ($0 \leq j \leq 10$) 情况下的决策结果。 $w_4=5, v_4=4$

	0	1	2	3	4	5	6	7	8	9	10
dp	0	0	6	6	9	9	9	10	11	13	14

$$temp[j] = \max\{dp[j], dp[j-w_4] + v_4\} \quad 0 \leq j \leq 10$$

	0	1	2	3	4	5	6	7	8	9	10
temp	0	0	6	6	9	9	9	10	11	13	14

	0	1	2	3	4	5	6	7	8	9	10
1	0	0	1	1	1	1	1	1	1	1	1
2	0	0	0	0	1	1	1	1	1	1	1
3	0	0	0	0	0	0	0	0	1	1	1
4	0	0	0	0	0	0	0	1	0	1	0
5											



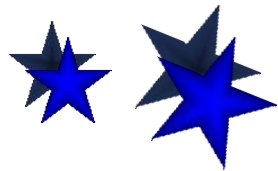
- * 第5阶段决策：前5个物品在背包容量不超过 j ($0 \leq j \leq 10$) 情况下的决策结果。 $w_5=4, v_5=6$

dp	0	1	2	3	4	5	6	7	8	9	10
	0	0	6	6	9	9	12	12	15	15	15

$$temp[j] = \max\{dp[j], dp[j-w_5] + v_5\} \quad 0 \leq j \leq 10$$

temp	0	1	2	3	4	5	6	7	8	9	10
	0	0	6	6	9	9	12	12	15	15	15

sel		0	1	2	3	4	5	6	7	8	9	10
	1	0	0	1	1	1	1	1	1	1	1	1
	2	0	0	0	0	1	1	1	1	1	1	1
	3	0	0	0	0	0	0	0	0	1	1	1
	4	0	0	0	0	0	0	0	1	0	1	0
	5	0	0	0	0	0	0	1	1	1	1	1



➤ 解的输出

(1) 满足约束条件的最大价值为 $dp[10]=15$

dp	0	1	2	3	4	5	6	7	8	9	10
	0	0	6	6	9	9	12	12	15	15	15

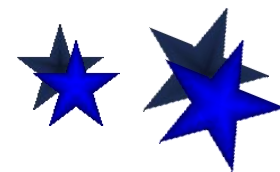
(2) 装入物品的输出

$sel[5][10]=1$, 说明装入物品5;

前4个物品的装入重量为 $10 - \text{物品5的重量}(4) = 6$

查看 $sel[4][6]=0$, 说明不装入物品4

sel		0	1	2	3	4	5	6	7	8	9	10
	1	0	0	1	1	1	1	1	1	1	1	1
	2	0	0	0	0	1	1	1	1	1	1	1
	3	0	0	0	0	0	0	0	0	1	1	1
	4	0	0	0	0	0	0	0	1	0	1	0
	5	0	0	0	0	0	0	1	1	1	1	1



sel

	0	1	2	3	4	5	6	7	8	9	10
1	0	0	1	1	1	1	1	1	1	1	1
2	0	0	0	0	1	1	1	1	1	1	1
3	0	0	0	0	0	0	0	0	1	1	1
4	0	0	0	0	0	0	0	1	0	1	0
5	0	0	0	0	0	0	1	1	1	1	1

物品4不装，说明前3个物品的容量为6，
查看`sel[3][6]=0`，说明不装入物品3；
继续查看`sel[2][6]=1`，说明装入物品2
前1个物品的容量=6-物品2的重量(2)=4
查看`sel[1][4]=1`，说明装入物品1。

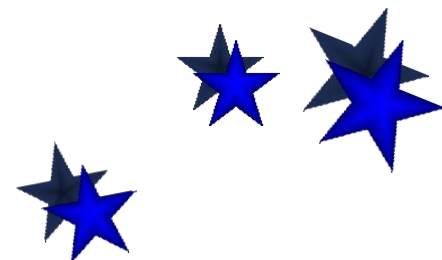
```
i=n,j=W;  
while(i>0)  
    if(sel[i][j]==1) { printf("i");i--,j=j-w[i];}  
    else i--;
```

3.3 动态规划算法的典型应用

◆ **完全背包问题**：一个旅行者准备随身携带一个背包。可以放入背包的物品有 n 种, 每种物品的重量和价值分别为 w_j, v_j , **每种物品可以装入多个**。如果背包的最大重量限制是 b , 怎样选择放入背包的物品以使得背包的价值最大?

● 问题建模:

(1) 解是 $\langle x_1, x_2, \dots, x_n \rangle$, 其中 x_i 是装入背包的第 i 种物品个数, x_i 为 ≥ 0 的整数。



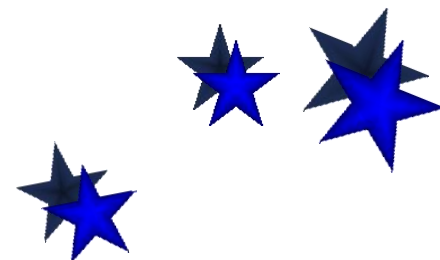
3.3 动态规划算法的典型应用

(2)

目标函数 $\max \sum_{i=1}^n v_i x_i$

约束条件 $\sum_{i=1}^n w_i x_i \leq b, \quad x_i \in \mathbb{N}$

- 线性规划问题: 由线性条件约束的线性函数取最大或最小的问题。
- 整数规划问题: 线性规划问题的变量 x_i 都是非负整数。



3.3 动态规划算法的典型应用

- 阶段划分

- 根据物品的个数，将问题的求解划分成 n 个阶段
- 第 k 个阶段决策出前 k 个物品在背包容量不超过 y 时的最大价值，记作： $F_k(y)$ ($0 \leq k \leq n$, $0 \leq y \leq b$)

- 子问题的边界

- $F_0(y)=0$ 不装入物品时，背包价值为0
- $F_k(0)=0$ 背包容量为0时，背包价值为0
- $F_k(y)=-\infty$ ($y < 0$) 背包容量为负数时，背包价值为 $-\infty$

- 状态转移方程

- 在决策 k 阶段，容量为 y 的最大价值时， $k-1$ 阶段以及 k 阶段小于 y 的子问题均已经解决。



3.3 动态规划算法的典型应用

➤ 递推方程1: $F_k(y) = \max \{F_{k-1}(y), F_k(y-w_k) + v_k\}$

没有装入
物品 k

k 阶段已对物品 k
决策完的结果。

说明: $F_k(y-w_k)$

(1) $y-w_k < 0$, 说明当前的 y 不够装入物品 k , 此时其值
 $F_k(y) = F_{k-1}(y)$ 。

(2) $y-w_k \geq 0$, 说明当前的 y 能够装入物品 k , 此时装入物品 k , 剩余容量 $y-w_k$ 分两种情况:

- 未曾装入物品 k , 则 $F_k(y-w_k) = F_{k-1}(y-w_k)$
- 已经装入过物品 k , 则 $F_k(y-w_k) \geq F_{k-1}(y-w_k)$
- 上述两种情况可统一为: $F_k(y-w_k) + v_k$



3.3 动态规划算法的典型应用

➤ 递推方程2(形式同资源分配问题)

$$F_k(y) = \max_{0 \leq x_k \leq \lfloor y/w_k \rfloor} \{F_{k-1}(y - x_k w_k) + x_k v_k\}$$

说明:

- (1) x_k 为物品 k 的装入个数, 从0直到 $\text{int}(y/w_k)$, 取价值最大
- (2) 写法1和写法2含义相同, 但效率相差很大
 - 写法1: 计算 $F_k(y)$ 时, $F_{k-1}(y)$ 和 $F_k(y-w_i)$ 已经计算完毕, 直接取值使用, 因此复杂度为 $O(1)$ 。
 - 写法2: 要经过 $\text{int}(y/w_k)$ 次循环才能得出 $F_k(y)$ 的值, 因此若 y 较大, w_k 较小时, 计算量也是很客观的。



3.3 动态规划算法的典型应用

- 解的追踪

(1) 使用一维数组 dp 存储第 k 阶段的决策结果。在第 k 阶段可先将决策结果暂存于 $temp$ 数组，该阶段结束后再更新 dp 数组中的内容。

(2) 使用二维数组 sel 实现。 $sel[k, y]$ 存储 k 阶段容量不超过 y 且取得最大价值时，**装入的最后一个物品编号**。例如：

- $sel[3,4]=2$ ，说明在第3阶段容量不超过4且价值最大时，最后装入的物品是2，即此时没有装入3号物品。
- $sel[4,8]=4$ ，说明在第4阶段容量不超过8且价值最大时，最后装入的物品是4。

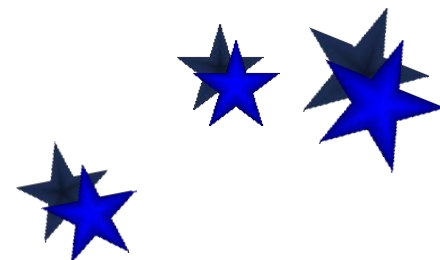


3.3 动态规划算法的典型应用

– $sel[k,y]$ 值的选取

$$sel[k,y] = \begin{cases} sel[k-1,y] & F_{k-1}(y) > F_k(y-w_k) + v_k \quad \text{没有装入物品 } k \\ k & F_{k-1}(y) \leq F_k(y-w_k) + v_k \quad \text{装入物品 } k \end{cases}$$

$$sel[1,y] = \begin{cases} 0 & y < w_1 \quad \text{装入物品 1} \\ 1 & y \geq w_1 \quad \text{装入物品 1} \end{cases}$$



3.3 动态规划算法的典型应用

- 实例 输入: $v_1=1, v_2=3, v_3=5, v_4=9,$
 $w_1=2, w_2=3, w_3=4, w_4=7,$
 $b=10$
- 数据结构
 - dp : 存储当前阶段求得的最大价值, 所有阶段结束后, $dp[10]$ 中即为最优值。
 - sel : 存储 k 阶段 y 容量取得最优值时, 装入的最后一个物品编号。
- 初始化

	0	1	2	3	4	5	6	7	8	9	10
dp	0	0	0	0	0	0	0	0	0	0	0



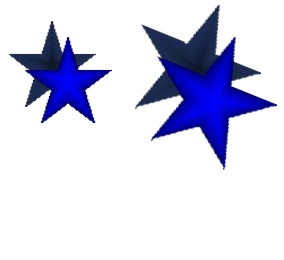
- * 第1阶段决策：前1个物品在背包容量不超过 j ($0 \leq j \leq 10$) 情况下的决策结果。 $w_1=2, v_1=1$

	0	1	2	3	4	5	6	7	8	9	10
dp	0	0	1	1	2	2	3	3	4	4	5

$$temp[j] = \max\{dp[j], \textcolor{red}{temp}[j-w_1] + v_1\} \quad 0 \leq j \leq 10$$

	0	1	2	3	4	5	6	7	8	9	10
temp	0	0	1	1	2	2	3	3	4	4	5

	0	1	2	3	4	5	6	7	8	9	10
1	0	0	1	1	1	1	1	1	1	1	1
2											
3											
4											



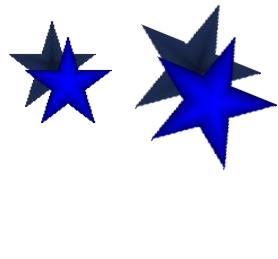
- * 第2阶段决策：前2个物品在背包容量不超过 j ($0 \leq j \leq 10$) 情况下的决策结果。 $w_2=3, v_2=3$

dp	0	1	2	3	4	5	6	7	8	9	10
	0	0	1	3	3	4	6	6	7	9	9

$$temp[j] = \max\{dp[j], temp[j-w_2] + v_2\} \quad 0 \leq j \leq 10$$

temp	0	1	2	3	4	5	6	7	8	9	10
	0	0	1	3	3	4	6	6	7	9	9

	0	1	2	3	4	5	6	7	8	9	10
1	0	0	1	1	1	1	1	1	1	1	1
2	0	0	1	2	2	2	2	2	2	2	2
3											
4											



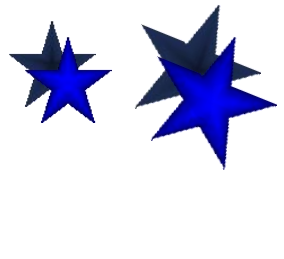
- * 第3阶段决策：前3个物品在背包容量不超过 j ($0 \leq j \leq 10$) 情况下的决策结果。 $w_3=4, v_3=5$

	0	1	2	3	4	5	6	7	8	9	10
dp	0	0	1	3	5	5	6	8	10	10	11

$$temp[j] = \max\{dp[j], temp[j-w_3] + v_3\} \quad 0 \leq j \leq 10$$

	0	1	2	3	4	5	6	7	8	9	10
temp	0	0	1	3	5	5	6	8	10	10	11

	0	1	2	3	4	5	6	7	8	9	10
1	0	0	1	1	1	1	1	1	1	1	1
2	0	0	1	2	2	2	2	2	2	2	2
3	0	0	1	2	3	3	3	3	3	3	3
4											



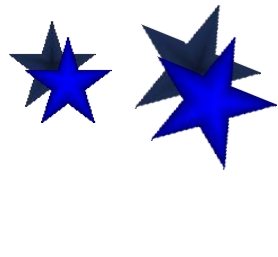
- * 第4阶段决策：前4个物品在背包容量不超过 j ($0 \leq j \leq 10$) 情况下的决策结果。 $w_4=7, v_4=9$

dp	0	1	2	3	4	5	6	7	8	9	10
	0	0	1	3	5	5	6	9	10	10	12

$$temp[j] = \max\{dp[j], temp[j-w_4] + v_4\} \quad 0 \leq j \leq 10$$

temp	0	1	2	3	4	5	6	7	8	9	10
	0	0	1	3	5	5	6	9	10	10	12

	0	1	2	3	4	5	6	7	8	9	10
1	0	0	1	1	1	1	1	1	1	1	1
2	0	0	1	2	2	2	2	2	2	2	2
3	0	0	1	2	3	3	3	3	3	3	3
4	0	0	1	2	3	3	3	4	3	4	4



➤ 解的输出

(1) 满足约束条件的最大价值为 $dp[10]=12$

	0	1	2	3	4	5	6	7	8	9	10
dp	0	0	1	3	5	5	6	9	10	10	12

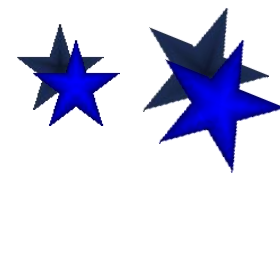
(2) 装入物品的输出

$sel[4][10]=4$, 说明最后一个装入物品4;

因物品4可能多次装入,

继续检查 $sel[4][10-w_4]=sel[4][3]$

	0	1	2	3	4	5	6	7	8	9	10
1	0	0	1	1	1	1	1	1	1	1	1
2	0	0	1	2	2	2	2	2	2	2	2
3	0	0	1	2	3	3	3	3	3	3	3
4	0	0	1	2	3	3	3	4	3	4	4





	0	1	2	3	4	5	6	7	8	9	10
1	0	0	1	1	1	1	1	1	1	1	1
2	0	0	1	2	2	2	2	2	2	2	2
3	0	0	1	2	3	3	3	3	3	3	3
4	0	0	1	2	3	3	3	4	3	4	4

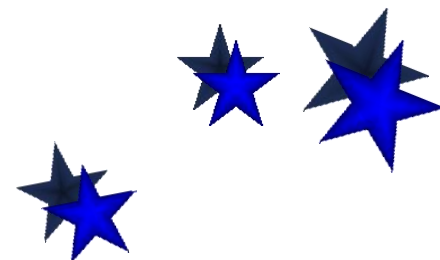


查看 $sel[4][3]=2$ ，说明该状态下，最后装入的物品编号是2号，即4号只装了1次、3号没有装。

因为2号物品也可能装入多个，

因此继续查看 $sel[2][3-w_2]=sel[2][0]=0$ ，

解的追踪结束。



根据 sel 矩阵追踪解的算法

算法3.3 TrackSolution

输入: sel 表

输出: $x_1, x_2, \dots, x_n$, n 种物品的装入量

1. for $j=1$ to n do

2. $x_j \leftarrow 0$

将解向量初始化为0

3. $y \leftarrow b, j \leftarrow n$

4. $j \leftarrow sel[j, y]$

从 $sel[n, b]$ 开始追踪解

5. $x_j \leftarrow 1$

6. $y \leftarrow y - w_j$

7. while $sel[j, y] = j$ do

8. $y \leftarrow y - w_j$

物品 j 是否多次装入

9. $x_j \leftarrow x_j + 1$

10. if $sel[j, y] \neq 0$ goto 4

物品编号 $\neq 0$, 继续追踪



3.3 动态规划算法的典型应用

- 时间复杂度估算

根据公式

$$F_k(y) = \max\{F_{k-1}(y), F_k(y - w_k) + v_k\}$$

备忘录需计算 nb 项，每项常数时间，
计算时间为 $O(nb)$ 。

伪多项式时间算法：时间为参数 b 和 n 的多项式，不是输入规模的多项式。
参数 b 是整数，表达 b 需要 $\log b$ 位，
输入规模是 $\log b$ 。

